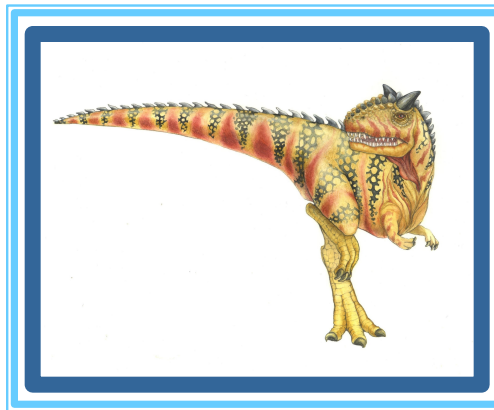


Process Management

Rakhshan Imtiaz

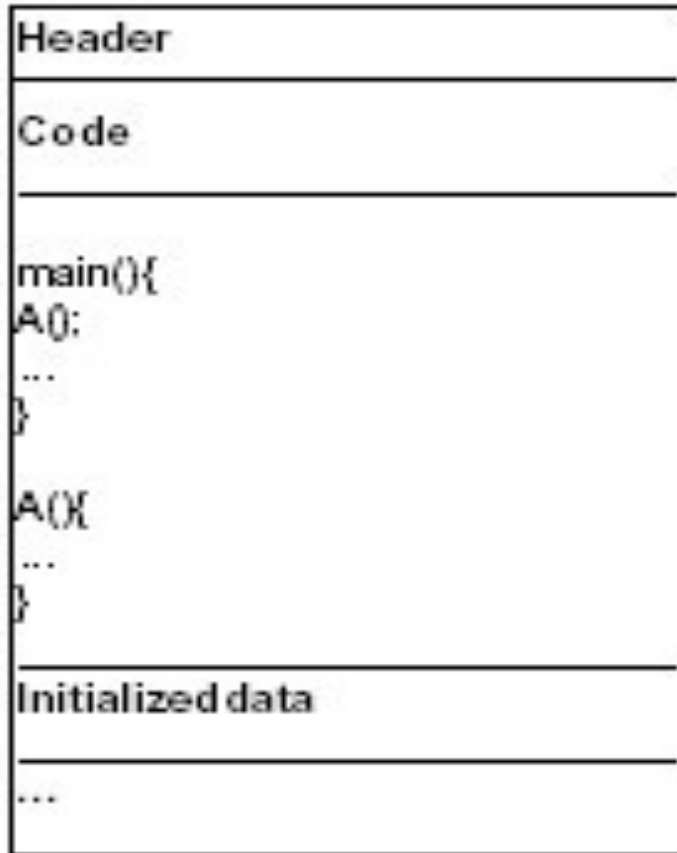




Process?=Program

Program

(e.g., executable file on disk)



- Program: series of commands (e.g. C statements, assembly commands, shell commands)





Process

Process:

- A Program in execution on machine is called process.

More Formally:

- A Sequential stream of Execution in its own address space





Process

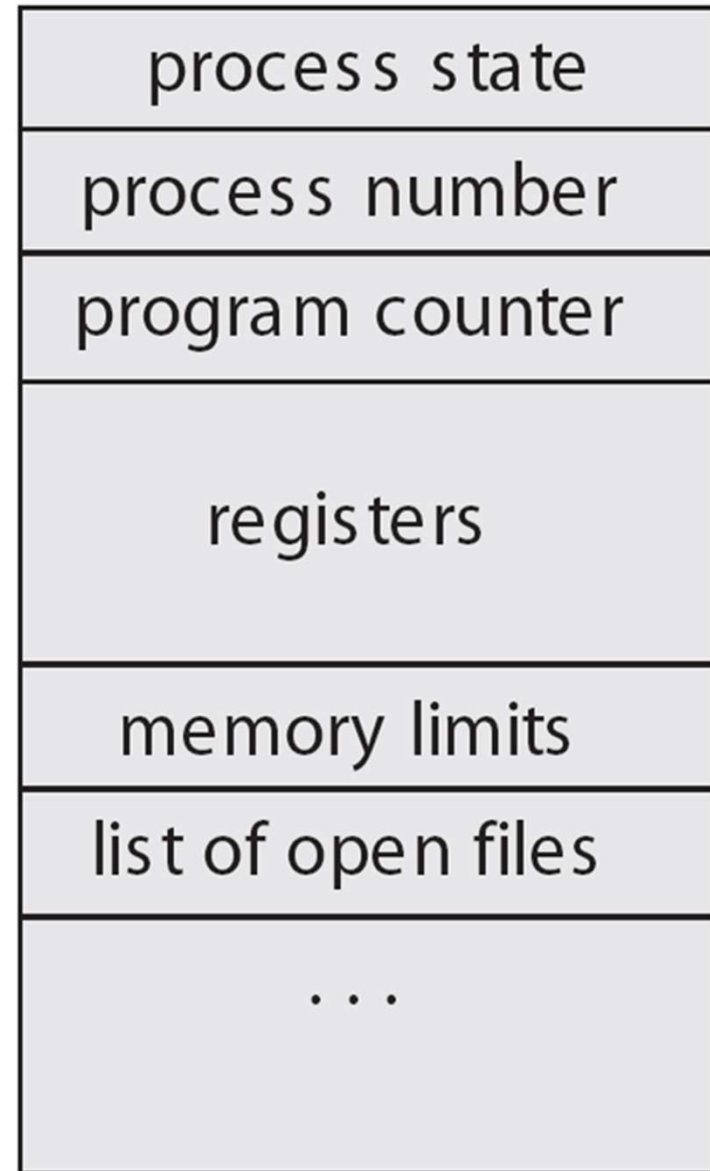
- A process consists of
 - Code (text) section
 - Data section
 - Stack
 - Heap
 - CPU State (program counter, etc.)
 - Environment
 - Process control block (PCB)





Process Control Block (PCB)

- The OS keeps all the data it needs about a process in the process control block (PCB)
- Thus another definition of a process:
 - “the entity described by a PCB”





Process Control Block (PCB)

- PCB is "the manifestation of a process in an operating system".
- **Data Structure** defined in the operating system kernel containing the information needed to manage a particular process.
- It must be kept in an area of memory protected from normal user access.





Process Control Block (PCB)

- PCB contains the Process information and attributes
 - Process state
 - Program counter
 - CPU registers
 - CPU scheduling information
 - Memory management information
 - Accounting information
 - I/O status information
 - Per process file table
 - Process ID (PID)





How process state changes_1

Ready

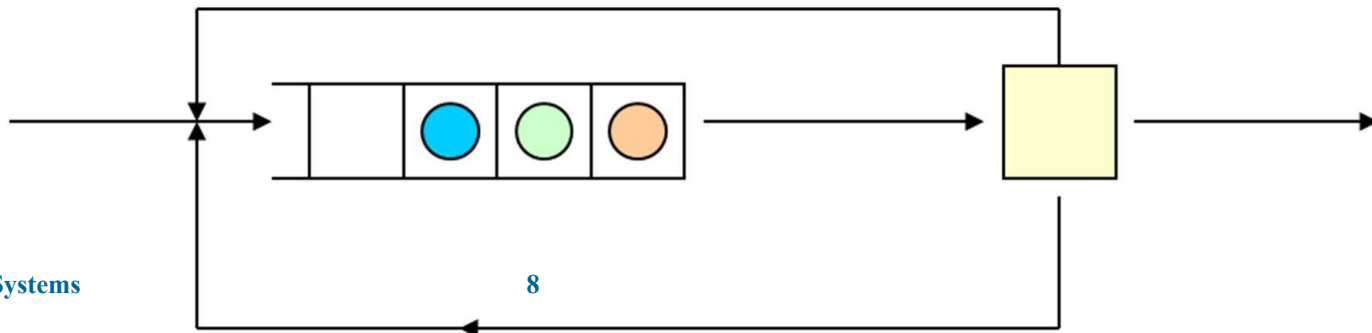
```
→ a := 1
  b := a + 1
  c := b + 1
  read a file
  a := b - c
  c := c * b
  b := 0
```

Ready

```
→ a := 1
  read a file
  b := a + 1
  c := b + 1
  a := b - c
  c := c * b
  b := 0
```

Ready

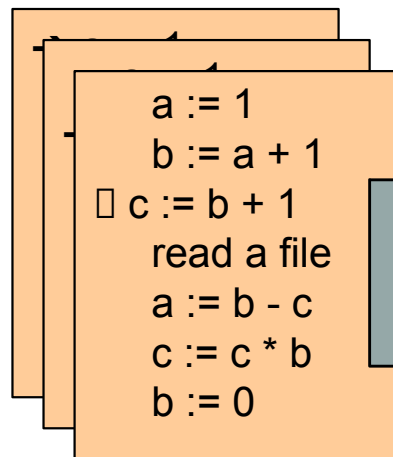
```
→ a := 1
  b := a + 1
  c := b + 1
  a := b - c
  c := c * b
  b := 0
  c := 0
```



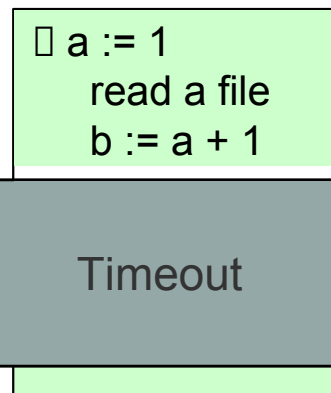


How process state_2

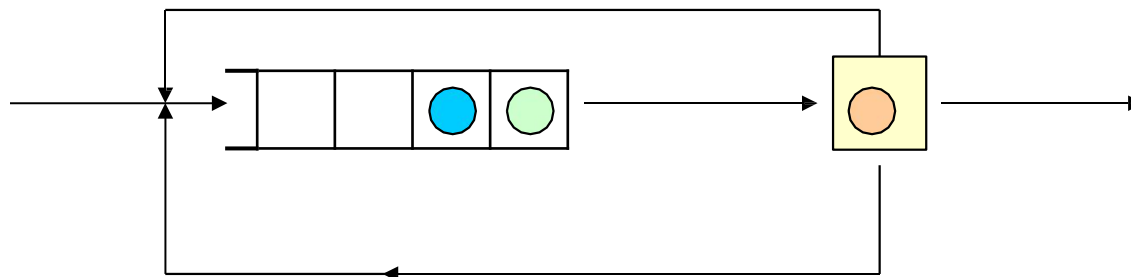
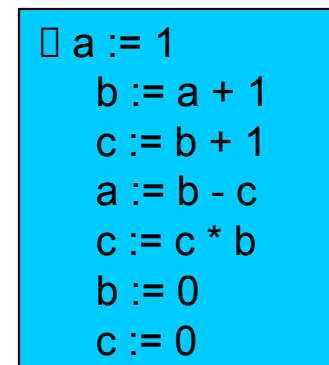
Running



Ready

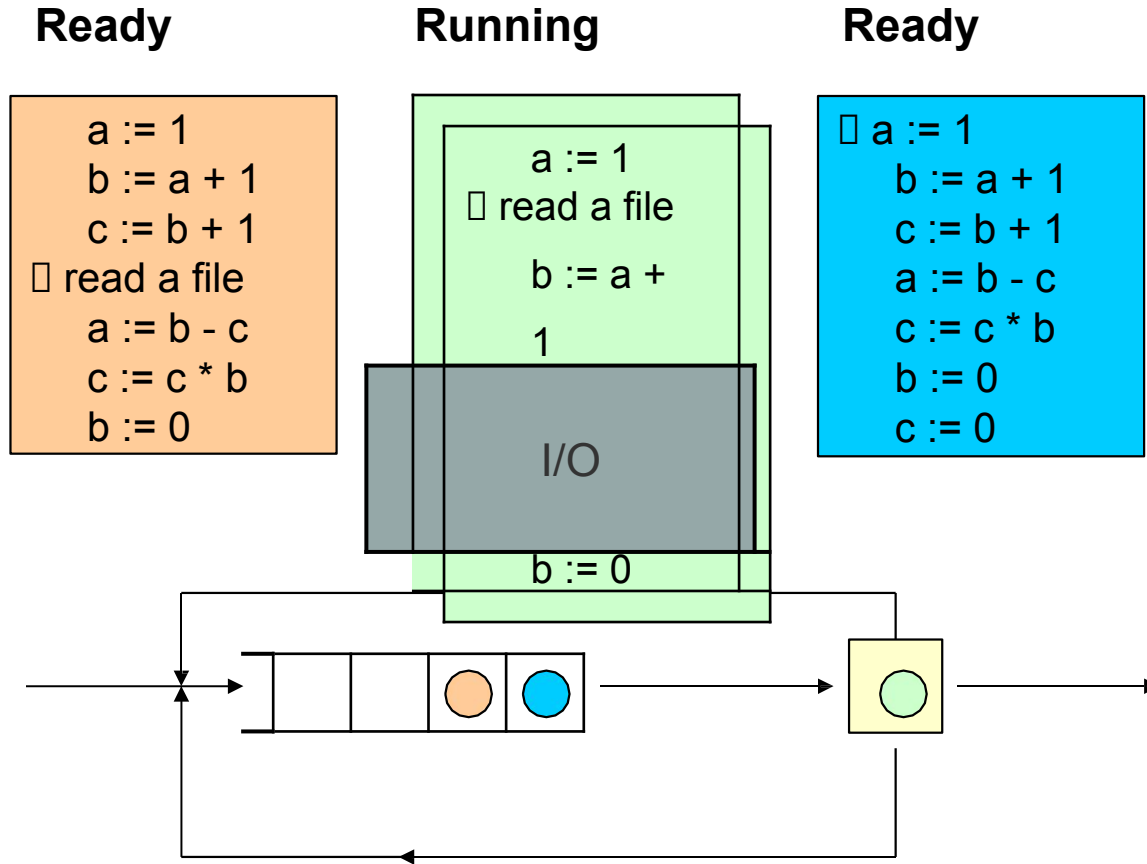


Ready





How process state changes_3





How process state changes_4

Ready

```
a := 1
b := a + 1
c := b + 1
□ read a file
a := b - c
c := c * b
b := 0
```

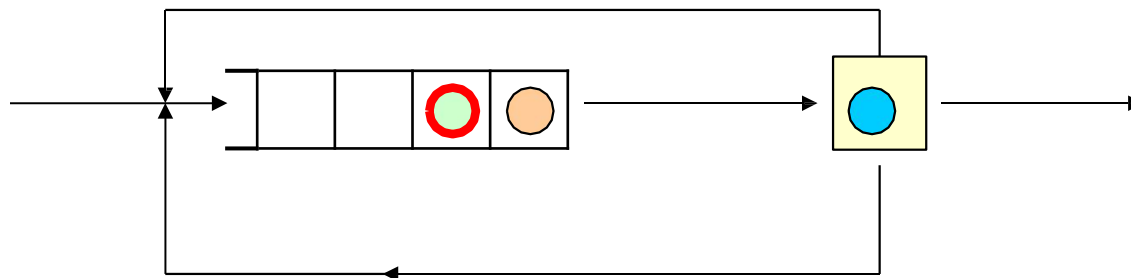
Blocked

```
a := 1
□ read a file
b := a + 1
```

Timeout

Running

```
a := 1
b := a + 1
□ c := b + 1
a := b - c
c := c * b
b := 0
c := 0
```





How process state changes_5

Running

```
a := 1
b := a + 1
c := b + 1
□ read a file
a := b - c
c := c * b
b := 0
```

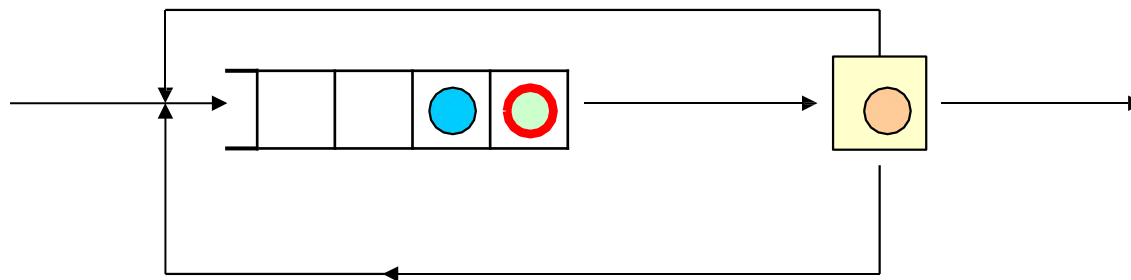
Blocked

```
a := 1
□ read a file
b := a + 1
```

I/O

Ready

```
a := 1
b := a + 1
c := b + 1
□ a := b - c
c := c * b
b := 0
c := 0
```





How process state changes_6

Blocked

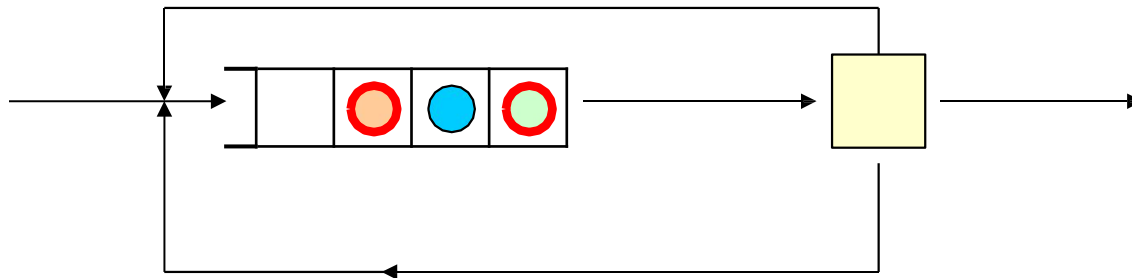
```
a := 1
b := a + 1
c := b + 1
□ read a file
a := b - c
c := c * b
b := 0
```

Blocked

```
a := 1
□ read a file
b := a + 1
c := b + 1
a := b - c
c := c * b
b := 0
```

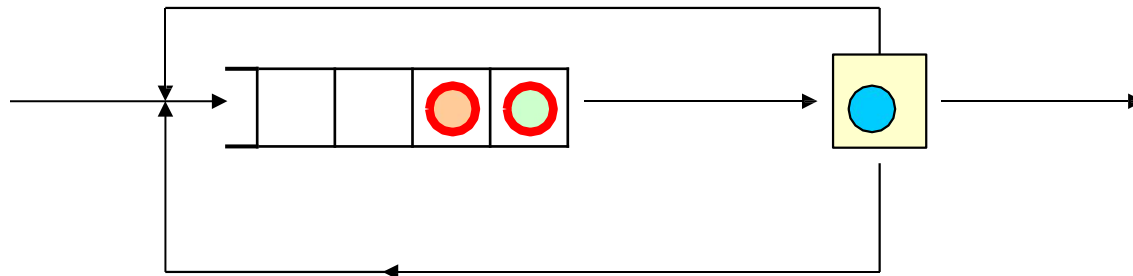
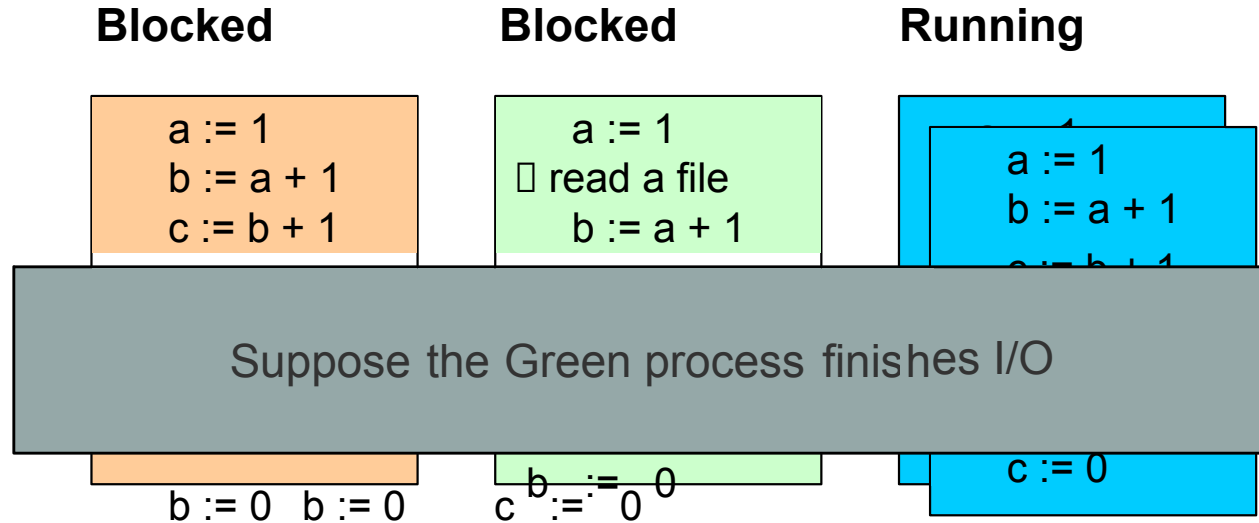
The Next Process to Run cannot be simply selected from the front

```
□ a := b - c
  := c * b
b := 0
c := 0
```





How process state changes_7





How process state changes_8

Blocked

```
a := 1
b := a + 1
c := b + 1
□ read a file
a := b - c
c := c * b
b := 0
```

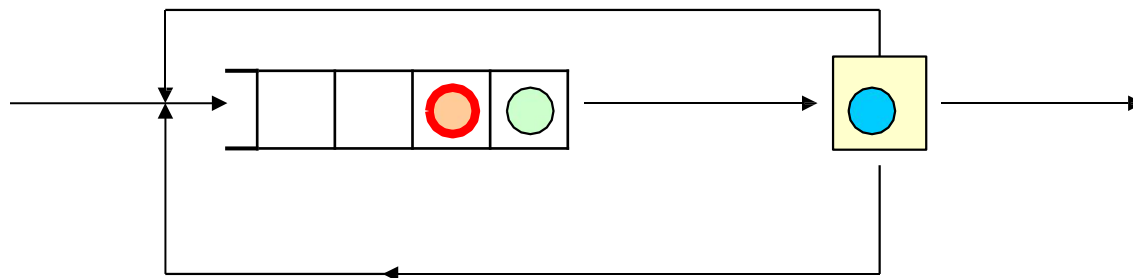
Ready

```
a := 1
read a file
□ b := a + 1
```

Timeout

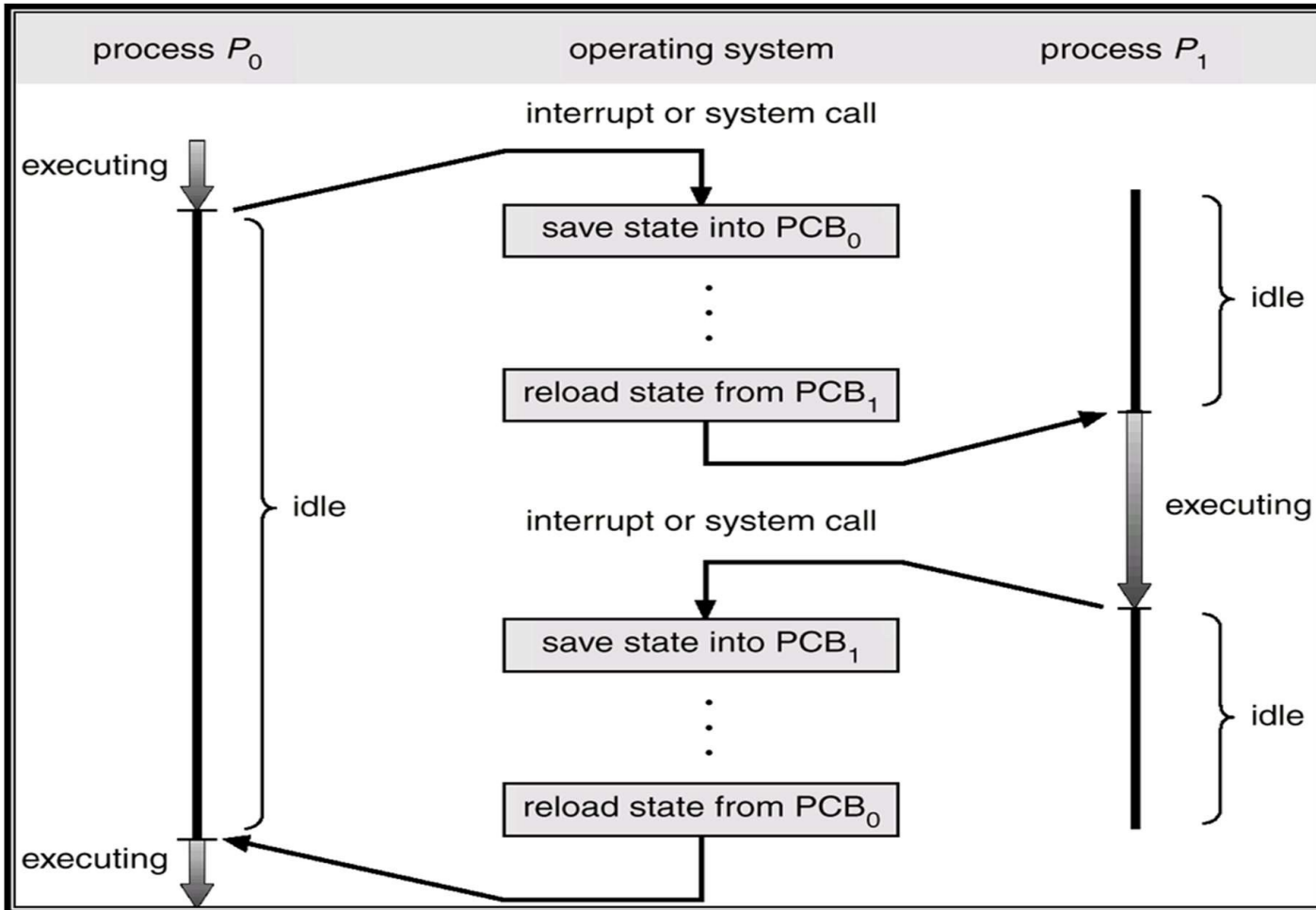
Running

```
a := 1
b := a + 1
c := b + 1
a := b - c
c := c * b
□ b := 0
c := 0
```





CPU Switch From Process to Process





CPU Switch From Process to Process

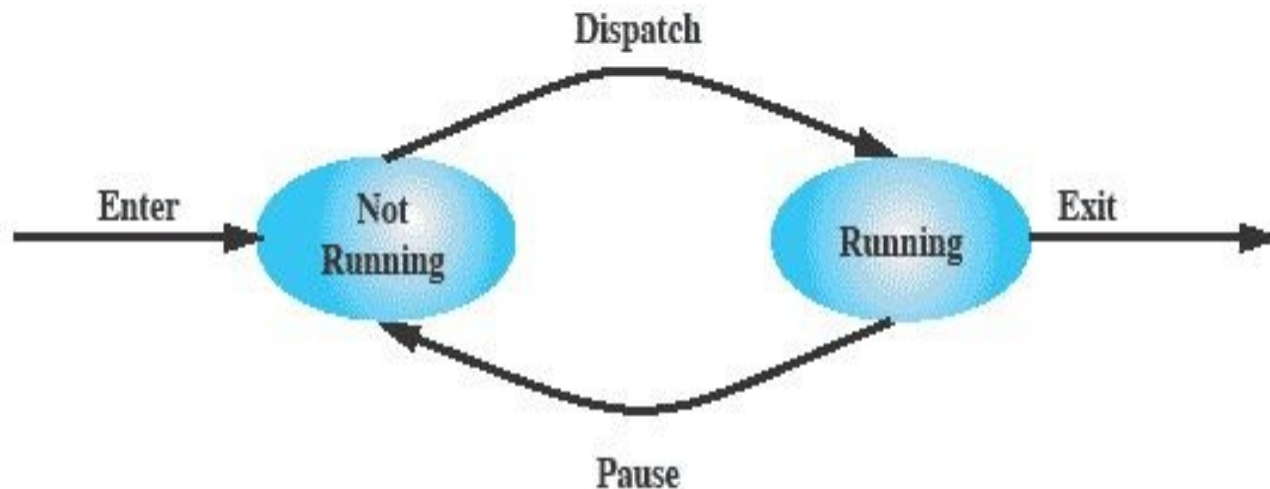
- Switching a process requires
 - Saving the state of old process
 - Loading the saved state of the new process
- This is called **Context Switch**
- Part of OS responsible for switching the processor among the processes is called **Dispatcher**





Two-State Process Model

- Process may be in one of two states
 - Running
 - Not-running

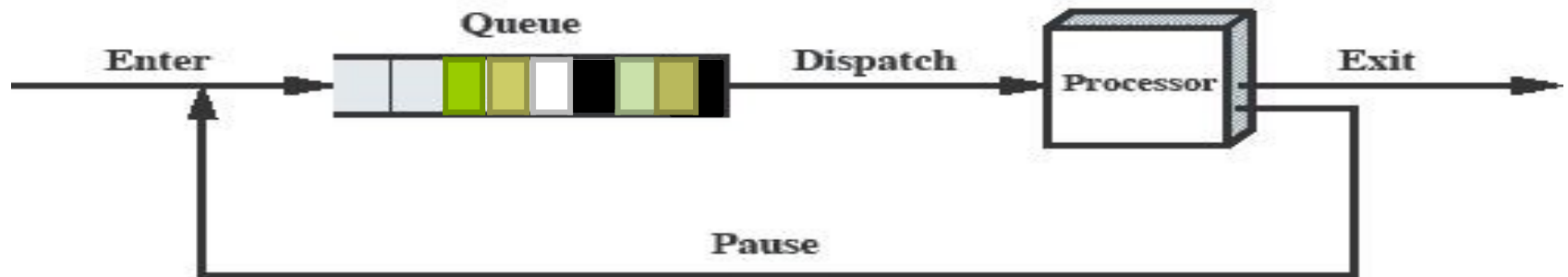


(a) State transition diagram





Queuing Diagram



(b) Queuing diagram





Five-state

Process Model

- The **state** of a process is defined by the current activity of that process.
- Each process may be in one of the following states

Running: currently being run

Ready: ready to run

Blocked: waiting for an event (I/O)

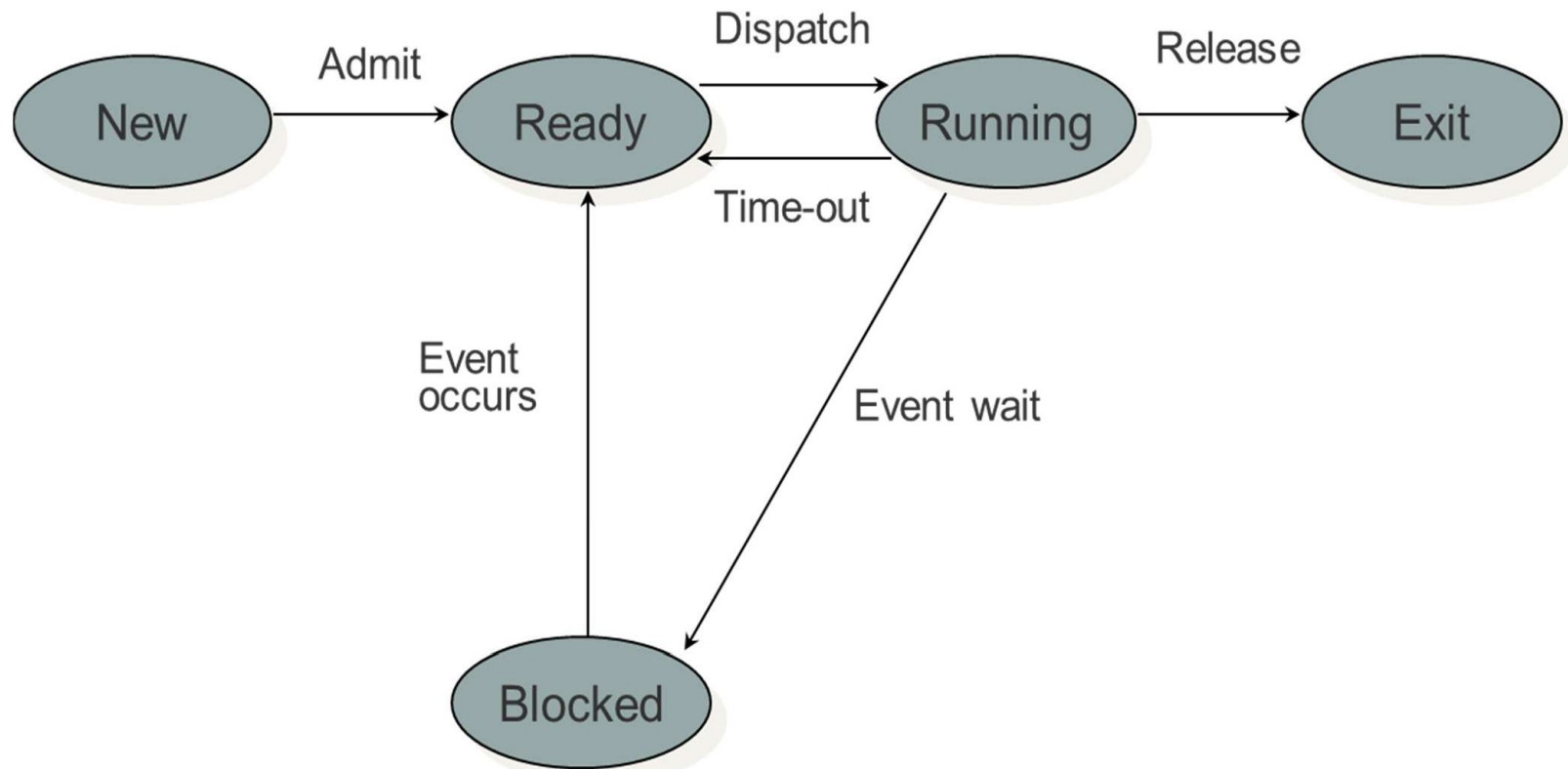
New: just created.

Exit: completed



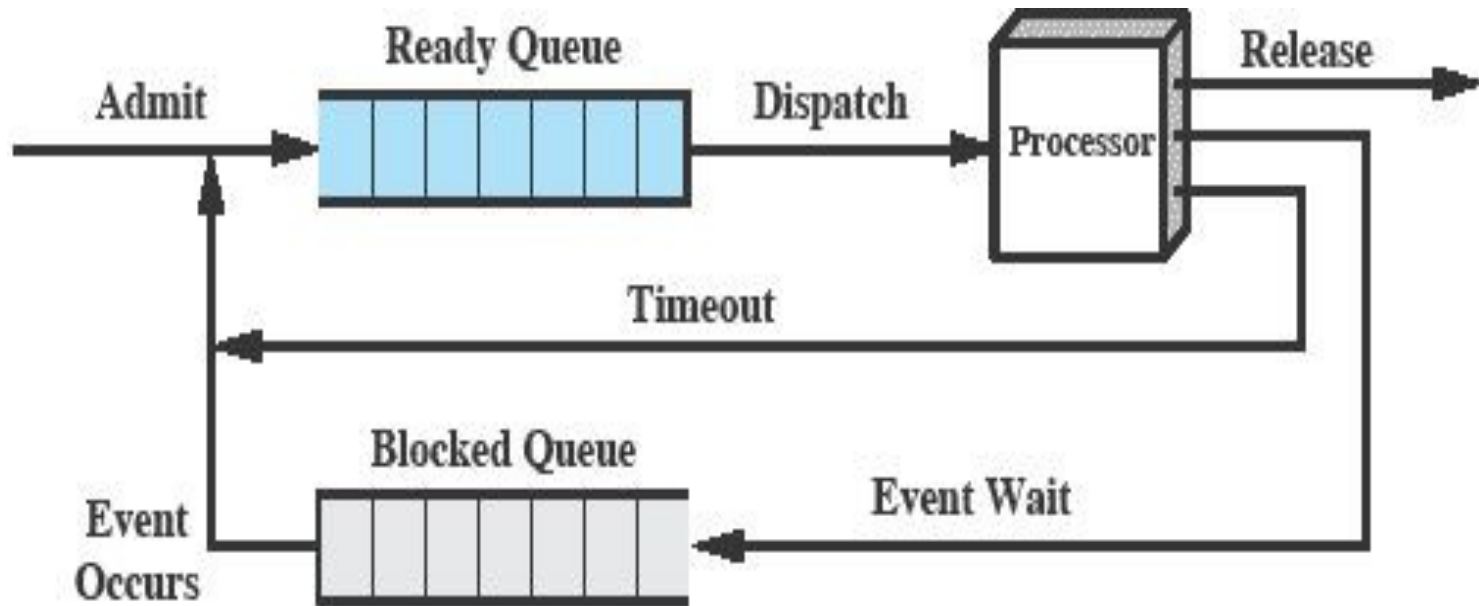


Five-state Process Model





Using Two Queues



(a) Single blocked queue



END

